

AI Critique (200–300 words)

The clearest place the AI got something wrong was its default assumption that “checklist execution” could be satisfied by reading source code alone. When first asked to build and run a GUI checklist, the model happily produced Pass/Fail verdicts without flagging, until directly pressed, that it had never actually rendered the application in a browser — a code-reading pass can catch logic bugs (an XSS sink, a doubled revenue calculation) but cannot catch genuine *rendered* usability problems like awkward spacing, a button that’s technically present but visually buried, or color contrast that fails only at certain zoom levels. This is a systematic blind spot, not a one-off mistake: static analysis is biased toward what’s textually inspectable and blind to what only emerges at render time or under real user interaction, which is exactly what GUI/usability testing is supposed to evaluate. The model caught this only when asked to be explicit about its method, not proactively.

A second failure was an early temptation, quickly caught, to fill the usability-evaluation participant table with “example” names to make the deliverable look complete. That the assignment’s own anti-cheat clause explicitly forbids this made it easy to catch, but it revealed a deeper tendency: language models default toward *appearing* thorough over being honestly incomplete, because plausible-looking placeholder data reads as more finished than a blank template with a TODO.

The principle I’m taking from this: an AI collaborator should be directed to state its verification method (code-read vs. live-tested) as a first-class output, not an afterthought, and any field requiring real-world provenance (people, screenshots, timestamps) should default to conspicuously blank rather than plausibly filled. Notably, the fix for the first failure was not to discard the code-informed findings — the bugs themselves were real — but to make the AI re-express each one as an observed action-and-result on the running app (what was typed/clicked and what happened), and to drop the one item that had no observable form at all. That distinction, between using code knowledge to *design* a test versus using it as the test’s evidence, is the actual skill a tester needs and the AI did not apply by default.